Case Study (Andhra Pradesh (AP) Assembly election results)

DATASET CURATION

Curated a dataset for Andhra Pradesh (AP) elections covering the years 2014, 2019, and 2024. A dummy dataset was generated using the OpenAI API. Since there are 175 Assembly Constituencies (ACs), each sheet contains 175 rows.

The columns contain - Assembly Constituency (AC) number / name - Party-wise vote counts - Total votes polled in each AC

Example Datapoints:

AC_No	AC_Name	TDP	YSRCP	INC	BJP	JSP	Total_Votes
1	AC_1	34592	23278	56048	52098	49256	215272
2	AC_2	38289	33434	31395	75302	24165	202585
3	AC_3	23905	32280	48657	50495	86237	241574
4	AC_4	23478	46062	74987	48893	78878	272298
5	AC_5	56463	20851	40926	75392	64597	258229
6	AC_6	56421	40379	48221	64118	33396	242535
7	AC_7	32156	69797	32676	67052	65082	266763

Dataset Description:

COLUMN	TYPE	DESCRIPTION
AC_NO	Integer	Unique assembly constituency number (1–175)
AC_NAME	String	Name of the assembly constituency
TDP	Integer	Votes polled for Telugu Desam Party
YSRCP	Integer	Votes polled for YSR Congress Party
INC	Integer	Votes polled for Indian National Congress
BJP	Integer	Votes polled for Bharatiya Janata Party
JSP	Integer	Votes polled for Jana Sena Party
TOTAL_VOTES	Integer	Total valid votes polled in the constituency

CHOICE OF LLM

For output generation, I used the **GPT-OSS 120B** model (From OpenAI), an open-source instruction-tuned model with tool-calling capabilities, which performed very well for this use case. **Groq provides free API access to the hosted model with a 200K tokens-per-day limit,** which was sufficient for the current scope of this project.

SYSTEM PROMPT

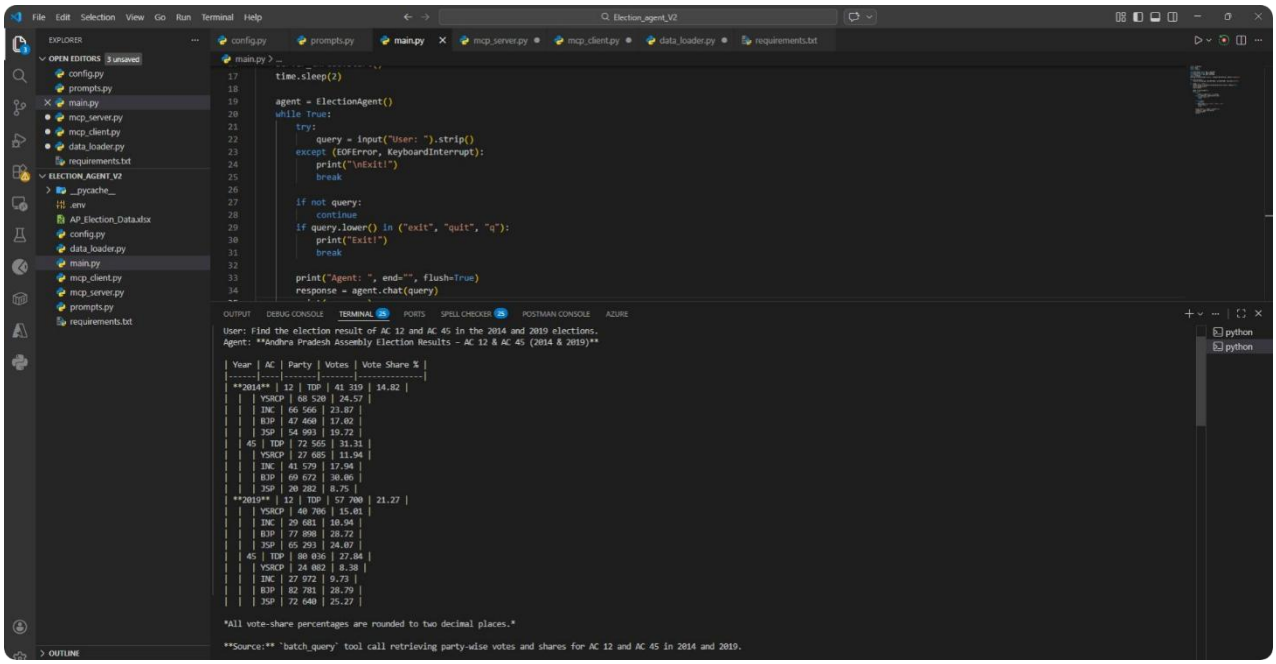
```
SYSTEM_PROMPT = """
You are a deterministic, fact-only election analysis AI for Andhra Pradesh (AP)
Assembly Elections (years: 2014, 2019, 2024).

Rules:
1. NEVER fabricate, estimate, or assume any number.
2. EVERY data point MUST come from an MCP tool call — no exceptions.
3. If a tool returns {"error": "..."}, relay the exact error to the user.
4. If data for a query does not exist in the dataset, say explicitly:
 "Data not found in dataset."
5. Always respond in structured, clearly labelled format (tables when useful).
6. Round vote shares to 2 decimal places in your final answer.

TOOL ROUTING GUIDE:
Use the mcp_tool with the appropriate tool_name:
Full results for one/many ACs in a year -> get_ac_results
Party vote share in one AC -> get_party_vote_share
Who won an AC in a year -> get_winner
Vote swing for a party across ACs & years -> compute_vote_swing
All elections compared for one AC -> compare_across_elections
Top/bottom constituencies by party votes -> get_top_constituencies
State-wide party summary for a year -> get_state_party_summary
Look up AC number from a partial name -> search_constituency_by_name
Multiple ACs × parties × years at once -> batch_query

KNOWN PARTIES:
TDP, YSRCP, INC, BJP, JSP
Party names in tool calls must be UPPERCASE.

AC NUMBER LOOKUP:
If the user provides an AC name (not number), call
search_constituency_by_name first to resolve the AC_NO, then proceed.

Response format guidelines:
- Use a markdown table for multi-AC or multi-year comparisons.
- Lead with the direct answer, then provide supporting numbers.
- End with a one-line "Source:" noting the tool(s) called.
"""
```

AVAILABLE TOOLS

get_winner	Winning party, vote count, and vote share for a single AC and year
get_party_vote_share	A specific party's votes and percentage share in one AC
compute_vote_swing	Change in a party's vote share between two years across multiple ACs
compare_across_elections	Side-by-side comparison of all election years for one AC
get_top_constituencies	Top or bottom N ACs ranked by a party's vote share in a year
get_state_party_summary	State-wide aggregate: total votes, seat count, and vote share per party
search_constituency_by_name	Resolve an AC number from a partial or full constituency name
batch_query	Multi-AC × multi-party × multi-year query in a single round-trip

RESULTS

All the sample questions provided in the case study pdf has been tested and validated.



DETERMINISTIC OUTPUT

For getting deterministic outputs I have set **temperature parameter of the LLM was set to 0 and sampling as false.** Which makes the results deterministic. In the output we can see that a slight change is there in the last line (2-3 tokens) but the rest is exactly same. So the results are **deterministic and reproducible**.

```
OUTPUT    DEBUG CONSOLE    TERMINAL 25    PORTS    SPELL CHECKER 25    POSTMAN CONSOLE    AZURE

**Source:** `get_party_vote_share` (AC 78, INC, 2024) and `get_party_vote_share` (AC 78, INC, 2019).

User: Which party won AC 101 in all three elections?
Agent: **Answer:** No single party won AC 101 in all three elections. The constituency was won by different parties in each election year.

| Election Year | Winning Party |
|---------------|---------------|
| 2014 | YSRCP |
| 2019 | BJP |
| 2024 | YSRCP |

*Source: `compare_across_elections` tool call.*

User: Which party won AC 101 in all three elections?
Agent: **Answer:** No single party won AC 101 in all three elections. The constituency was won by different parties in each election year.

| Election Year | Winning Party |
|---------------|---------------|
| 2014 | YSRCP |
| 2019 | BJP |
| 2024 | YSRCP |

**Source:** `compare_across_elections` (AC 101).

User:
```

FEATURES

- **Multiturn Conversation –** The system handles multi-turn conversations.
- **Persistent Conversation Memory -** The LLM has Memory across tool responses and can summarize and generate insights from previous queries.
- **Dynamic Data Handling -** The system handles data integration dynamically.
- **Safety -** Safety is accounted for in the code in two places.
  1. The first layer of safety is forced using the **System prompt**.
  2. The second layer is handled by the **MCP_Server** which checks for proper formatting of data, datatypes and mismatches.

SCALING FRAMEWORK FOR PRODUCTION

- The xl dataset is currently being processed using pandas the dataset can be dynamically transformed into a structured SQL format and stored in a relational database like **PostgreSQL or MySQL**. This would enable faster indexing, optimized querying, and more efficient search operations. That will result in significantly reducing latency as the data grows or concurrent queries increase.
- To efficiently handle multiple user request **Asynchronous processing** can be implemented. We can use **async I/O** so that the system can **process concurrent queries** without blocking execution and decrease latency of the system.
- To manage and optimize API usage costs, a **rate limiter** can be introduced.
- A **multi-agent architecture** can be introduced where different agents are responsible for specialized tasks and tool calls. Where a **verification or validation agent** can be added as a final layer. This agent would cross-check numerical outputs, validate consistency and ensure the response aligns with the structured data returned by tools. This will help with **hallucinations** as well. (Although the latency of the system might increase a bit but based on the clint requirement this can be implemented)